



nextwork.org

Website Delivery with CloudFront



Arpita Chowdhury

User Information

☐



Introducing Today's Project!

In this project, I will demonstrate creation of S3 bucket and upload website files, set up a CloudFront to distribute website globally, manage permission for both S3 and CloudFront and compare and analyze performance for different methods for hosting website. I'm doing this project to learn to learn about website delivery with CloudFront.

Tools and concepts

The services I used were Amazon S3 for storing and hosting my website files and Amazon CloudFront for securely delivering the content through a global CDN. Key concepts I learned include content delivery networks (CDNs), origin access control (OAC), bucket policies, public vs. private access, and how caching improves website performance and reduces latency.

Project reflection

This project took me approximately 2 hours. The most challenging part was troubleshooting the AccessDenied errors and understanding how CloudFront permissions work differently from S3 static website hosting, especially with Origin Access Control (OAC). It was most rewarding to finally see both versions of my website load successfully and to understand the difference between public S3 hosting and secure CloudFront delivery.



Arpita Chowdhury

NextWork Student

nextwork.org

I chose to do this project today because I want to learn about CloudFront and continue progressing through the Cloud Engineering roadmap. Working through this lab helped me understand how content delivery, permissions, and performance optimization work in real AWS environments. Something that would make learning with NextWork even better is offering more support that helps candidates prepare for professional roles, such as guidance on real-world scenarios, best practices, and interview-focused skill development.



Set Up S3 and Website Files

I started the project by creating an S3 bucket to store and host my website files. I can't use CloudFront for this task because CloudFront is a content delivery network, not a storage service; it needs an origin like S3 to get the files from.

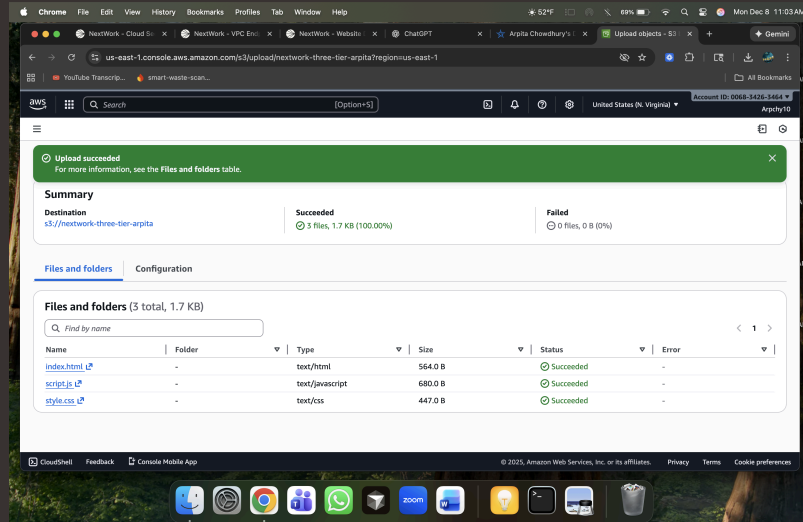
The three files that make up my website are: `index.html`, which provides the structure of the webpage, including the input field, action button, and section where user information is displayed; `style.css`, which controls the layout and visual appearance of the site by styling the text, input elements, and content container; and `script.js`, which adds functionality by sending a request to the API using the entered User ID and rendering the returned data on the page.

I validated that my website files work by running the site locally, inspecting the console for errors, and confirming that the HTML renders properly, the CSS styling is applied, and the JavaScript file retrieves and displays data from the API without issues.



Arpita Chowdhury
NextWork Student

nextwork.org





Exploring Amazon CloudFront

Amazon CloudFront is a content delivery network, which means it distributes your website's content through a global network of edge locations so users receive data from the server closest to them, resulting in faster load times and reduced latency. Businesses and developers use CloudFront because it improves website performance, enhances security through features like HTTPS and Origin Access Control, reduces load on the origin server, and provides a reliable, scalable way to deliver content to users anywhere in the world.

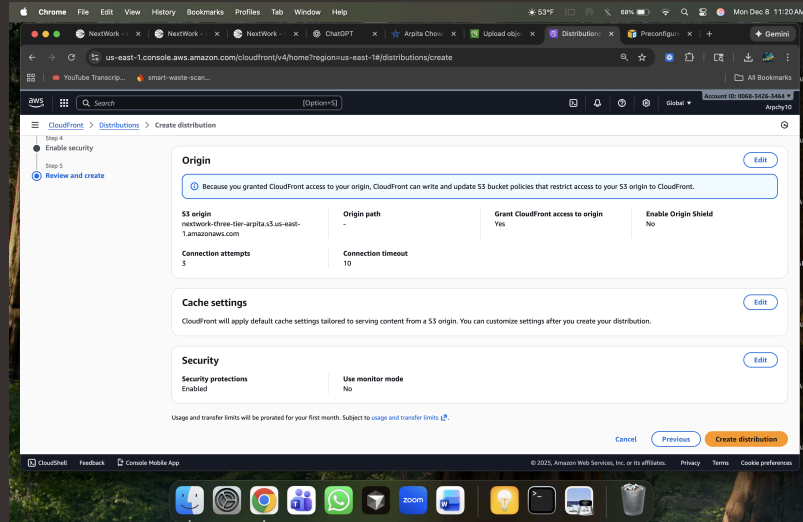
To use Amazon CloudFront, I set up distributions, which are configurations that tell CloudFront where content comes from (the origin), how it should be cached, and how it should be delivered securely to users around the world. I set up a distribution for my website so that CloudFront can retrieve the files stored in my S3 bucket, cache them at global edge locations, and deliver them to users with high performance and low latency. The origin is the S3 bucket that stores my website content, which CloudFront uses as the source location for all the files served through the distribution.

My CloudFront distribution's default root object is `index.html`. This means that when a user visits the root URL of my site without specifying a file name CloudFront will automatically serve the `index.html` file as the homepage of the website.



Arpita Chowdhury
NextWork Student

nextwork.org





Handling Access Issues

When I tried visiting my distributed website, I ran into an `AccessDenied` error because my S3 bucket is private and I hadn't fully configured the bucket policy to allow access only through my CloudFront distribution's Origin Access Control (OAC), so the origin rejected the request.

My distribution's origin access settings were public. This caused the access denied error because it does not automatically guarantee that CloudFront is authorized to read objects from the S3 bucket the bucket itself must explicitly allow CloudFront access through an Origin Access Control (OAC) or similar mechanism. Without the correct bucket policy, S3 blocks the request and returns `AccessDenied`.

To resolve the error, I set up Origin Access Control (OAC). OAC is a CloudFront feature that securely authorizes CloudFront to access a private S3 bucket by signing requests and ensuring that only CloudFront not the public can retrieve the content stored in the bucket.



Arpita Chowdhury

NextWork Student

nextwork.org

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8" ?>
<Error>
  <Code>AccessDenied</Code>
  <Message>Access Denied</Message>
</Error>
```



Updating S3 Permissions

Once I set up my OAC, I still needed to update my bucket policy because S3 must explicitly allow access from the CloudFront distribution using that OAC. Without updating the policy, CloudFront is authorized in principle, but the S3 bucket will still block the requests since it does not yet trust the specific CloudFront signing identity.

Creating an OAC automatically gives me a policy I could copy, which grants CloudFront permission to access the objects in my S3 bucket using its signed requests, while still keeping the bucket private from the public.

Policy

```
1 {
2   "Version": "2008-10-17",
3   "Id": "PolicyForCloudFrontPrivateContent",
4   "Statement": [
5     {
6       "Sid": "AllowCloudFrontServicePrincipal",
7       "Effect": "Allow",
8       "Principal": {
9         "Service": "cloudfront.amazonaws.com"
10      },
11      "Action": "s3:GetObject",
12      "Resource": "arn:aws:s3:::nextwork-three-tier-arpita/*",
13      "Condition": {
14        "StringEquals": {
15          "AWS:SourceArn": "arn:aws:cloudfront::006834263464:distribution/E1W223H45T2WVD"
16        }
17      }
18    }
19  ]
20 }
```



S3 vs CloudFront for Hosting

For my project extension, I'm comparing how my website behaves when hosted directly through S3 Static Website Hosting versus when delivered through CloudFront using Origin Access Control (OAC). I initially had an error with static website hosting because my S3 bucket was still configured to block all public access, which is required for CloudFront + OAC but incompatible with S3's website endpoint. Since static website hosting only works when the bucket objects are publicly readable, S3 returned a 403 Forbidden error when I tried to open the website URL.

I tried resolving this by unchecking the block all public access box in the permission page. I still ran into an error because unchecking block all public access doesn't grant permission to access the objects. It simply stops blocking all public access attempts.

I could finally see my S3 hosted website when I edited my bucket policy to grant permission to distribute website. This worked because I added this statement to my bucket policy to make S3 static website hosting work because it grants public read access (s3:GetObject) to all objects in the bucket, which is required for the S3 website endpoint to serve files without returning a 403 error.

placeholder

Compared to the permission settings for my CloudFront distribution, using S3 meant I had to make my bucket objects publicly readable, since the S3 website endpoint cannot use OAC or private access controls. This made the bucket openly accessible to anyone on the internet. I preferred the CloudFront setup because it allowed me to keep my S3 bucket fully private while still delivering the website securely.



Arpita Chowdhury

NextWork Student

nextwork.org

With OAC, only my CloudFront distribution could access the bucket, which provided better security and more control over how the content was served.



S3 vs CloudFront Load Times

Load time means the amount of time it takes for a webpage to fully appear and become usable in a user's browser after they request it. It reflects how quickly the server delivers the website files and how efficiently the browser can load them. The load times for the CloudFront site were faster than the S3 site because CloudFront uses a global network of edge locations that cache copies of my website closer to users. This reduces latency and avoids sending every request all the way back to the S3 bucket's region.

A business would prefer CloudFront when it needs faster global performance, stronger security, and more control over how its content is delivered, since CloudFront uses a worldwide network of edge locations and can securely access a private S3 bucket through OAC. In contrast, S3 static website hosting might be sufficient when the website is small, simple, low-traffic, or mainly accessed by users near the S3 region, because it provides an easy and inexpensive way to host a static site even though it requires the bucket to be publicly accessible and lacks the performance and security benefits of CloudFront.



User Information

Name	Status	Type	Initiator	Size	Time
nextwork-three-tier-arpita.s3-website-...	304	document	Other	0.3 kB	62 ms
style.css	200	stylesheet	/index.i2	(memory ...)	0 ms
script.js	200	script	/index.i6	(memory ...)	0 ms



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

